
QMNIST Dataset Image Understanding

Cher Li

Department of Computer Engineering
McGill University
cher.li@mail.mcgill.ca

Nara Yun

Department of Software Engineering
McGill University
nara.yun@mail.mcgill.ca

Karen Cherry Chen Lai

Department of Computer Engineering
McGill University
karen.chenlai@mail.mcgill.ca

Abstract

This project addresses a 10-class image classification task involving digits presented within the QMNIST dataset. A supervised model is implemented using PyTorch, employing dataset-specific normalization and data augmentation techniques including random rotation, affine transformations, horizontal flipping, and color jittering, with only random rotation retained in the final configuration to enhance model generalization. A custom Dataset class was designed to efficiently manage data loading and transformations. The model was trained and validated, with final predictions on a test dataset submitted to a Kaggle competition for evaluation. The final public result is 0.98833, and the model involves custom normalization and data transformation through random rotation, and a custom CNN class with AdamW optimizer.

1 Introduction

This project focuses on developing a machine learning system for image classification. The dataset consists of 0-9 digit classes, with each image containing a single digit and outside characters from the QMNIST dataset. The primary objective is to build an accurate supervised classification model capable of digit recognition.

The available data includes labeled training images (Train.pkl with corresponding Train_labels.csv) and unlabeled test images (Test.pkl). Model performance is evaluated through a Kaggle competition framework, where participants submit predicted labels for the test set.

Our implementation utilizes a PyTorch-based deep learning pipeline incorporating several key components:

- Dataset-specific preprocessing and normalization (normalized using computed mean and standard deviation)
- Comprehensive data augmentation testing random rotation, affine transformations, horizontal flipping, and color jittering (settling on only random rotation of 15 degrees)
- Custom convolution neural network (custom residual blocks combining 2D convolution layers and ReLU activation, custom fully connected layer of linear transformations) alongside the AdamW optimizer

This work demonstrates the effectiveness of systematic neural network approaches for noisy image classification tasks, highlighting the importance of proper data preprocessing, augmentation, and model training techniques in achieving accuracy and performance.

2 Dataset and Preprocessing

The dataset comprises of grayscale images with singular channels and dimensions 28×28 pixels, each displaying a digit alongside supplementary characters from the QMNIST dataset. The training set, stored in `Train.pkl`, includes 60,000 annotated images, with corresponding numerical labels (0-9) provided in `Train_labels.csv`. These labels are represented in scientific notation (e.g., $5E + 00$). For evaluation purposes, `Test.pkl` contains unlabeled test images requiring classification.

This configuration establishes a 10-class recognition challenge, where the principal objective involves accurately classifying the central digit in each test image and submitting predictions in the specified CSV format for Kaggle competition evaluation.

The data preparation protocol involves two critical phases [1]:

- Initial normalization using training-set-derived mean and standard deviation parameters
- Comprehensive augmentation of training samples including:
 - Geometric transformations (random rotations, affine distortions, horizontal flips)
 - Photometric variations (brightness/contrast adjustments)

Table 1: Data Preprocessing Accuracy

Epoch	ToTensor (baseline)	ToPILImage	Normalize (computed)	Normalize (0.5)
1	0.9373	0.8559	0.9062	0.9282
2	0.9640	0.9607	0.9622	0.9654
3	0.9519	0.9698	0.9625	0.9647
4	0.9519	0.9566	0.9664	0.9697
5	0.9663	0.9718	0.9723	0.9674

As the number of epoch increases, the accuracy also generally increases, but for the sake of the report accuracies will be reported using epoches under 10. The final model was trained for 30 epoches.

The accuracy results below are conducted with mean and standard deviation equaling 0.5 even if the final model utilizes a custom normalization as it shows higher accuracy. This also demonstrates how the combination of different features, even if each has the best accuracy out of the selected topic, may not be the highest accuracy overall. This is a topic for future investigations.

Table 2: Geometric and Color Transformation Accuracy

Epoch	RandomRotation (15)	RandomHorizontalFlip	RandomAffine	ColorJitter
6	0.8878	0.5275	0.5229	0.8493
7	0.9371	0.7889	0.6522	0.8681
8	0.9492	0.8572	0.7678	0.8712
9	0.9479	0.9058	0.7728	0.8869
10	0.9526	0.9179	0.8561	0.8876

3 Proposed Approach

3.1 Model Development Process

To implement the improved CNN model, a systematic development process was followed, focusing on effective feature extraction, stable training, and robust generalization.

The feature extraction module consists of three convolutional blocks, each containing two convolutional layers with batch normalization and ReLU activation. Max pooling is applied after each block to downsample spatial dimensions while preserving essential features. Batch normalization reduces internal covariate shift, thereby improving training stability and allowing higher learning rates [2]. ReLU activation introduces non-linearity and mitigates the vanishing gradient problem [3].

To address potential degradation in deeper architectures, a residual connection was introduced. Specifically, a convolutional shortcut connects the output of the first block to the second, enabling the model to learn residual mappings rather than direct mappings. This promotes stable training by improving gradient flow and facilitates the construction of deeper networks [4].

After feature extraction, the classification module processes the output feature maps by flattening them and passing them through a fully connected layer with ReLU activation. A final fully connected layer maps the features to class logits for prediction.

During preprocessing, input images were first converted to PIL format and subjected to random rotations of up to 15 degrees. This augmentation technique was used to improve generalization by exposing the model to rotational variations. The images were then converted to tensors and normalized using dataset-specific mean and standard deviation, effectively scaling pixel values to approximately the range $[-1, 1]$. Other augmentation methods such as horizontal flipping, affine transformations, and color jittering were not applied in the final training configuration.

The model was trained using the AdamW optimizer with a learning rate of 10^{-3} and a weight decay of 10^{-5} to encourage generalization [5]. Dynamic learning rate scheduling was implemented using a ReduceLROnPlateau scheduler, which lowered the learning rate when validation loss plateaued, improving convergence [6]. Cross-entropy loss was used as the objective function for multi-class classification. Training was conducted over 30 epoches with a batch size of 64.

To explore design choices, we conducted a series of ablation studies, adjusting key model components and recording validation performance.

Table 3 compares the impact of modifying the fully connected (fc) layers. Using only a single nn.Linear layer yielded the highest accuracy, while additional ReLU activation or Dropout led to reduced performance.

Table 3: Adjustment of fc_layer

Epoch	Linear	ReLU	Dropout (0.5)
6	0.8792	0.7618	0.6235
7	0.8782	0.7624	0.6643
8	0.8952	0.7820	0.6737
9	0.9006	0.7845	0.6769
10	0.8920	0.7861	0.6764

We also varied the residual block configuration, as shown in Table 4. Using both Conv2D and ReLU within the residual path slightly outperformed simpler residual structures, improving model stability and convergence.

Table 4: Adjustment of res_block

Epoch	None	Conv2D	ReLU	Conv2D + ReLU
6	0.9566	0.9575	0.9558	0.9535
7	0.9583	0.9531	0.9573	0.9503
8	0.9587	0.9597	0.9562	0.9583
9	0.9602	0.9670	0.9657	0.9620
10	0.9612	0.9600	0.9602	0.9629

Finally, we compared different optimizers under a fixed learning rate of 10^{-3} . Table 5 summarizes the results. AdamW consistently outperformed other methods, including standard Adam, SGD, and RMSprop, demonstrating the benefit of decoupled weight decay regularization.

Table 5: Different optimizers

Epoch	AdamW	Adam	SGD	RMSprop
6	0.9535	0.9454	0.5105	0.9373
7	0.9503	0.9544	0.5708	0.9582
8	0.9583	0.9579	0.6407	0.9571
9	0.9620	0.9542	0.6861	0.9577
10	0.9629	0.9597	0.7268	0.9624

Results

Table 6: Final models

Model	Accuracy	Mean Accuracy	Standard Deviation	Public score
Original	0.9307	0.8677	0.1136	0.97600
Random Rotation	0.9747	0.9539	0.0317	0.98600
Custom mean and std	0.9762	0.9562	0.0421	0.98666
Custom <code>fc_layer</code>	0.9765	0.9641	0.0166	0.98833

The final model involves custom data normalization, random rotation by 15 degrees, an adjustment of the `fc_layer` so only `nn.Linear` is present, an adjustment of the residual block through using both `Conv2D` and `ReLU`, and using AdamW for the optimizer.

Model performance is evaluated using both a validation set and the public test leaderboard on Kaggle. Validation accuracy is used as the primary metric during development and guided the selection of model architecture, data preprocessing strategies, and training configurations. Final performance is still assessed through Kaggle leaderboard submissions.

Initial experiments using 20 training epoches produced stable convergence, but increasing the number of epoches to 30 improves model generalization, yielding a final Kaggle accuracy of 0.988. This gain suggests that longer training provided better exposure to the augmented data, allowing the model to extract more robust features.

While the validation set serves as the internal benchmark for hyperparameter tuning, the final selected model achieves consistent performance on the Kaggle test set, demonstrating the effectiveness of the training strategy and generalization capability of the network.

4 Discussion and Conclusion

The classification task involves a structured approach that integrates both data preprocessing and custom model design. Preprocessing plays a vital role in preparing the data for effective learning. All input images are normalized using the computed mean and standard deviation of the dataset, ensuring consistency across samples. To enhance model generalization and robustness, random rotations of up to 15 degrees are applied as a data augmentation strategy. Other augmentation methods, including affine transformations, horizontal flipping, and color jittering, were also tested during development. However, these techniques either introduced unnatural distortions or disrupted digit orientation, leading to lower validation accuracy and reduced model stability. As such, they were excluded from the final configuration.

The model architecture is a custom convolutional neural network (CNN) inspired by established designs. It incorporates residual blocks that combine 2D convolutional layers with `ReLU` activations, allowing for deeper learning while mitigating the vanishing gradient problem. The fully connected layers are customized using Linear transformations, enabling the model to capture specific relationships in the data.

AdamW is selected as the optimizer due to its adaptive learning capabilities and built-in weight decay, which contribute to improved generalization and convergence. This combination of tailored

preprocessing, architectural adjustments, and an effective optimization strategy results in the strongest model performance observed during experimentation.

Ongoing investigation includes further exploration of hyperparameter tuning, such as experimenting with different learning rates and optimization strategies. Additional changes to the CNN architecture, such as incorporating novel layer designs, may also lead to performance improvements. As methods and models evolve, the ability to build more accurate and efficient models continues to expand. Such advancements not only improve performance in specific tasks but also contribute to the broader progress of the field.

Statement of Contributions

This project is a team effort, and each team member has contributed an equal amount of time to reach the current accuracy results.

Nara Yun: CNN implementation and hyperparameter tuning.

Cher Li: Data preprocessing, dataloader setup, and report writing.

Karen Cherry Chen Lai: Hyperparameter tuning, and report refinement.

References

- [1] A. Kochkorova and A. Wang, "Data Augmentation for Handwritten Digit Recognition," National High School Journal of Science, January 2025. <https://nhsjs.com/2025/data-augmentation-for-handwritten-digit-recognition/>
- [2] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," Proceedings of the 32nd International Conference on Machine Learning, 2015. <http://arxiv.org/abs/1502.03167>
- [3] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," Proceedings of the 27th International Conference on Machine Learning, 2010. <https://www.semanticscholar.org/paper/Rectified-Linear-Units-Improve-Restricted-Boltzmann-Nair-Hinton/a538b05ebb01a40323997629e171c91aa28b8e2f>
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. <https://ieeexplore.ieee.org/document/7780459>
- [5] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," International Conference on Learning Representations (ICLR), 2017. <https://www.semanticscholar.org/paper/Decoupled-Weight-Decay-Regularization-Loshchilov-Hutter/d07284a6811f1b2745d91bdb06b040b57f226882>
- [6] L. N. Smith, "Cyclical Learning Rates for Training Neural Networks," IEEE Winter Conference on Applications of Computer Vision (WACV), 2017. <https://ieeexplore.ieee.org/abstract/document/7926641>
- [7] P. Rone, "Building a CNN for Handwritten Digit Recognition with PyTorch: A Step-by-Step Guide," Medium, December 2024. <https://medium.com/@ponchanon.rone/building-a-cnn-for-handwritten-digit-recognition-with-pytorch-a-step-by-step-guide-9df1dcb1092d>

Appendix

4.1 Appendix A: Data Visualization

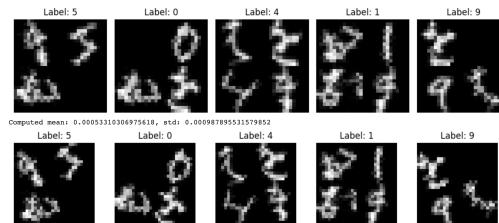


Figure 1: Original and Augmented Data

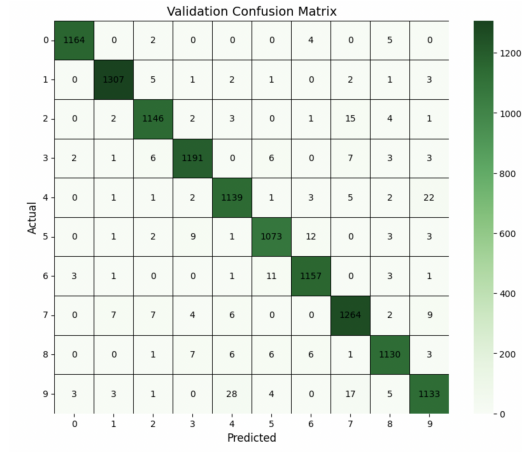


Figure 2: Confusion Matrix

4.2 Appendix B: Code

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, Subset
from torchvision import transforms
import pickle
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from google.colab import drive

drive.mount('/content/gdrive')
with open('/content/gdrive/MyDrive/ECSE551/Train.pkl', 'rb') as f:
    train_images = pickle.load(f)
    print("Train image shape:", train_images.shape)

with open('/content/gdrive/MyDrive/ECSE551/Test.pkl', 'rb') as f:
    test_images = pickle.load(f)
    print("Test image shape:", test_images.shape)
label_df = pd.read_csv('/content/gdrive/MyDrive/ECSE551/Train_labels.csv', header=None)
train_labels = label_df[0].astype(int).values

def preview_samples(images, labels, n=5):
    plt.figure(figsize=(10, 4))
    for i in range(n):
        img = images[i, 0, :, :]
        lbl = labels[i]
        plt.subplot(1, n, i + 1)
        plt.imshow(img, cmap='gray')
        plt.title(f"Label: {lbl}")
        plt.axis('off')
    plt.tight_layout()

plt.show()

preview_samples(train_images, train_labels, n=5)

def compute_mean_std(images):
    all_imgs = images.astype(np.float32) / 255.0
    mean = np.mean(all_imgs)
    std = np.std(all_imgs)
    return mean, std

mean, std = compute_mean_std(train_images)
print(f"Computed mean: {mean}, std: {std}")

# mean = 0.5
# std = 0.5

train_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomRotation(15),
    # transforms.RandomHorizontalFlip(),
    # transforms.RandomAffine(degrees=15, translate=(0.1, 0.1), shear=10),
    # transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize((mean,), (std,))
])

test_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.ToTensor(),
    transforms.Normalize((mean,), (std,))
])

class DigitDataset(Dataset):
    def __init__(self, images, labels=None, transform=None):
```

```

    self.images = images
    self.labels = labels
    self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img = self.images[idx].squeeze(0)
        if self.transform:
            img = self.transform(img)
        if self.labels is not None:
            return img, self.labels[idx]
        else:
            return img

train_dataset = DigitDataset(train_images, train_labels, transform=train_transforms)
test_dataset = DigitDataset(test_images, transform=test_transforms)

def show_augmented_images(dataset, count=5):
    plt.figure(figsize=(10, 2))
    for i in range(count):
        img_tensor, label = dataset[i]
        img_np = img_tensor.permute(1, 2, 0).numpy()
        img_np = img_np * 0.5 + 0.5 # De-normalize
        plt.subplot(1, count, i + 1)
        plt.imshow(img_np, cmap='gray')
        plt.title(f'Label: {label}')
        plt.axis('off')
    plt.tight_layout()
    plt.show()

show_augmented_images(train_dataset)

```

```

if torch.cuda.is_available():
    num_devices = torch.cuda.device_count()
    gpu_name = torch.cuda.get_device_name(0)
    print(f"GPU detected: {gpu_name} ({num_devices} CUDA device(s))")
    compute_device = torch.device("cuda")
else:
    print("No GPU detected. Using CPU for training.")
    compute_device = torch.device("cpu")

device = compute_device

```

```

class CustomConvNet(nn.Module): # To improve CNN
    def __init__(self, num_classes):
        super(CustomConvNet, self).__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            nn.MaxPool2d(kernel_size=2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.1),

            nn.MaxPool2d(kernel_size=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.MaxPool2d(kernel_size=2),
        )

```

```

        self.res_block = nn.Sequential(
            nn.Conv2d(128, 128, kernel_size=1),
            nn.ReLU()
        )

        self.fc_layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 3 * 3, 256),
            # nn.ReLU(),
            # nn.Dropout(p=0.5),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        out = self.conv_layers(x)
        shortcut = self.res_block(out)
        out = out + shortcut # Residual connection
        return self.fc_layers(out)

def run_training(model, dataloader, loss_fn, optimizer, device):
    model.train()
    total_loss = 0
    for imgs, lbls in dataloader:
        imgs, lbls = imgs.to(device), lbls.to(device)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = loss_fn(outputs, lbls)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(dataloader)

def validate_model(model, dataloader, device):

```

```

model.eval()
correct_preds = 0
total_samples = 0
with torch.no_grad():
    for imgs, lbls in dataloader:
        imgs, lbls = imgs.to(device), lbls.to(device)
        preds = model(imgs)
        _, predicted = torch.max(preds, 1)
        correct_preds += (predicted == lbls).sum().item()
        total_samples += lbls.size(0)
return correct_preds / total_samples

def collect_predictions(model, dataloader, device):
    model.eval()
    preds_all = []
    labels_all = []
    with torch.no_grad():
        for imgs, lbls in dataloader:
            imgs, lbls = imgs.to(device), lbls.to(device)
            preds = model(imgs)
            _, top_preds = torch.max(preds, 1)
            preds_all.extend(top_preds.cpu().numpy())
            labels_all.extend(lbls.cpu().numpy())
    return np.array(preds_all), np.array(labels_all)

class SmoothCrossEntropyLoss(nn.Module):
    def __init__(self, num_classes, smoothing=0.1):
        super(SmoothCrossEntropyLoss, self).__init__()
        self.num_classes = num_classes
        self.smoothing = smoothing
        self.confidence = 1.0 - smoothing

    def forward(self, predictions, targets):
        predictions = predictions.log_softmax(dim=-1)

```

```

        smoothed_targets = torch.zeros_like(predictions)
        smoothed_targets.fill_(self.smoothing / (self.num_classes - 1))
        smoothed_targets.scatter_(1, targets.unsqueeze(1), self.confidence)
        return torch.mean(torch.sum(-smoothed_targets * predictions, dim=-1))

[ ] from sklearn.model_selection import train_test_split

idx_train, idx_val = train_test_split( # 80/20 split for training and validation
    np.arange(len(train_dataset)),
    test_size=0.2,
    shuffle=True,
    random_state=42
)

subset_train = Subset(train_dataset, idx_train)
subset_val = Subset(train_dataset, idx_val)

loader_train = DataLoader(subset_train, batch_size=64, shuffle=True)
loader_val = DataLoader(subset_val, batch_size=64)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_classes = len(np.unique(train_labels))

model = CustomConvNet(num_classes=num_classes).to(device)

loss_fn = SmoothCrossEntropyLoss(num_classes=num_classes, smoothing=0.1)
optimizer = optim.Adam(model.parameters(), lr=0.001)
lr_scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', patience=3, factor=0.5
)

```

```

# Training Loop
val_scores = []

for ep in range(1, 31): # 30 total epochs
    epoch_loss = run_training(model, loader_train, loss_fn, optimizer, device)
    epoch_val_acc = validate_model(model, loader_val, device)
    lr_scheduler.step(epoch_val_acc)

    print(f"[Epoch {ep:02d}] | Loss: {epoch_loss:.4f} | Val Accuracy: {epoch_val_acc:.4f}")
    val_scores.append(epoch_val_acc)

```

```

predicted_vals, actual_vals = collect_predictions(model, loader_val, device)

# Generate confusion matrix
conf_mat = confusion_matrix(actual_vals, predicted_vals)
plt.figure(figsize=(10, 8))
heat = sns.heatmap(
    conf_mat,
    cmap="Greens",
    cbar=True,
    fmt="d",
    linewidths=0.5,
    linecolor="black"
)

for row in range(conf_mat.shape[0]):
    for col in range(conf_mat.shape[1]):
        val = conf_mat[row, col]
        heat.text(
            col + 0.5,
            row + 0.5,
            str(val),
            ha="center",
            va="center",
            color="black",
            fontsize=10
        )

plt.xlabel("Predicted", fontsize=12)
plt.ylabel("Actual", fontsize=12)
plt.title("Validation Confusion Matrix", fontsize=14)

tick_labels = np.arange(conf_mat.shape[0])
plt.xticks(tick_labels + 0.5, tick_labels, fontsize=10)
plt.yticks(tick_labels + 0.5, tick_labels, fontsize=10, rotation=0)

```



```
plt.show()

# Print Results Summary
print("Validation Summary:")
print(f"Validation Accuracies: {val_scores}")
print(f"Mean Accuracy: {np.mean(val_scores):.4f}")
print(f"Std Dev: {np.std(val_scores):.4f}")
```

```
train_all_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_classes = len(np.unique(train_labels))
model = CustomConvNet(num_classes=num_classes).to(device)
loss_fn = SmoothCrossEntropyLoss(num_classes=num_classes, smoothing=0.1)
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', patience=3, factor=0.5)

def train_entire_model(model, dataloader, criterion, optimizer, scheduler, epochs=30):
    for ep in range(epochs):
        model.train()
        epoch_loss = 0
        for x_batch, y_batch in dataloader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            preds = model(x_batch)
            loss = criterion(preds, y_batch)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
        avg_loss = epoch_loss / len(dataloader)
        scheduler.step(avg_loss)
        print(f"[Epoch {ep+1:02}] Full Training Loss: {avg_loss:.4f}")

print("Starting full training")
train_entire_model(model, train_all_loader, loss_fn, optimizer, scheduler)

def run_inference(model, dataloader):
    model.eval()
    all_preds = []
    with torch.no_grad():
        for batch in dataloader:
            batch = batch.to(device)
```

```
logits = model(batch)
_, top_classes = torch.max(logits, 1)
all_preds.extend(top_classes.cpu().numpy())
return np.array(all_preds)

print("Running inference on test data")
final_test_preds = run_inference(model, test_loader)

submission_df = pd.DataFrame({
    "id": np.arange(1, len(final_test_preds)+1),
    "class": final_test_preds.astype(int)
})
submission_df.to_csv("submission.csv", index=False)
print("Saved submission.csv (ready for Kaggle)")
```